



Git

Di cosa parleremo:

01 Perché serve il controllo di versione

03 I branch: sviluppo in parallelo

05 GitHub e repository remoti

02 Perché Git

04 Combinare i branch

06 Conflitti, fork e pull request

01

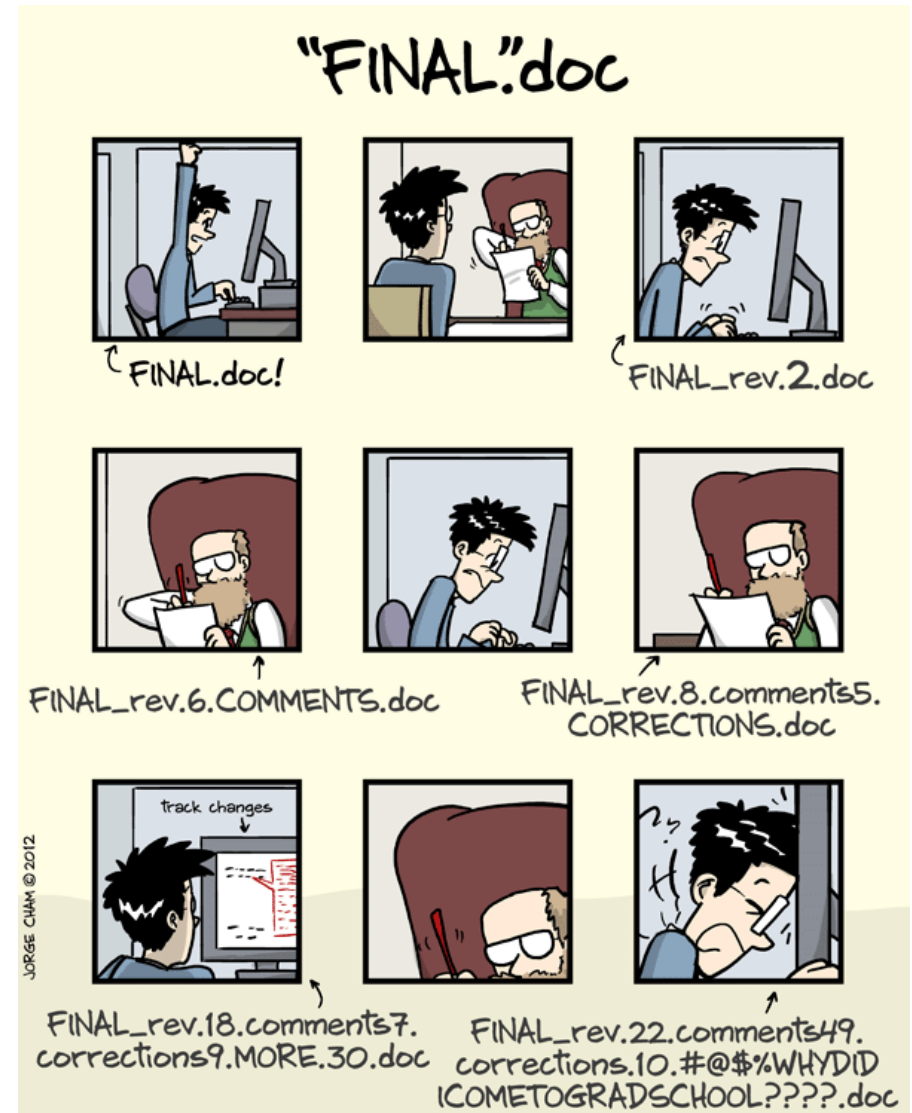
Perché serve il controllo di versione

01 • Version Control System (VCS)

Un software che gestisce le versioni dei tuoi file

Ti permette di rivedere versioni vecchie e di recuperare file cancellati per sbaglio:

- Delle modifiche “rompono” qualcosa, e si vuole tornare indietro;
- Si vuole provare qualcosa di nuovo, con la possibilità di annullarlo se non funziona;
- Si vuole condividere il nostro lavoro con qualcuno.



02

Perché Git

Modi per versionare i file:

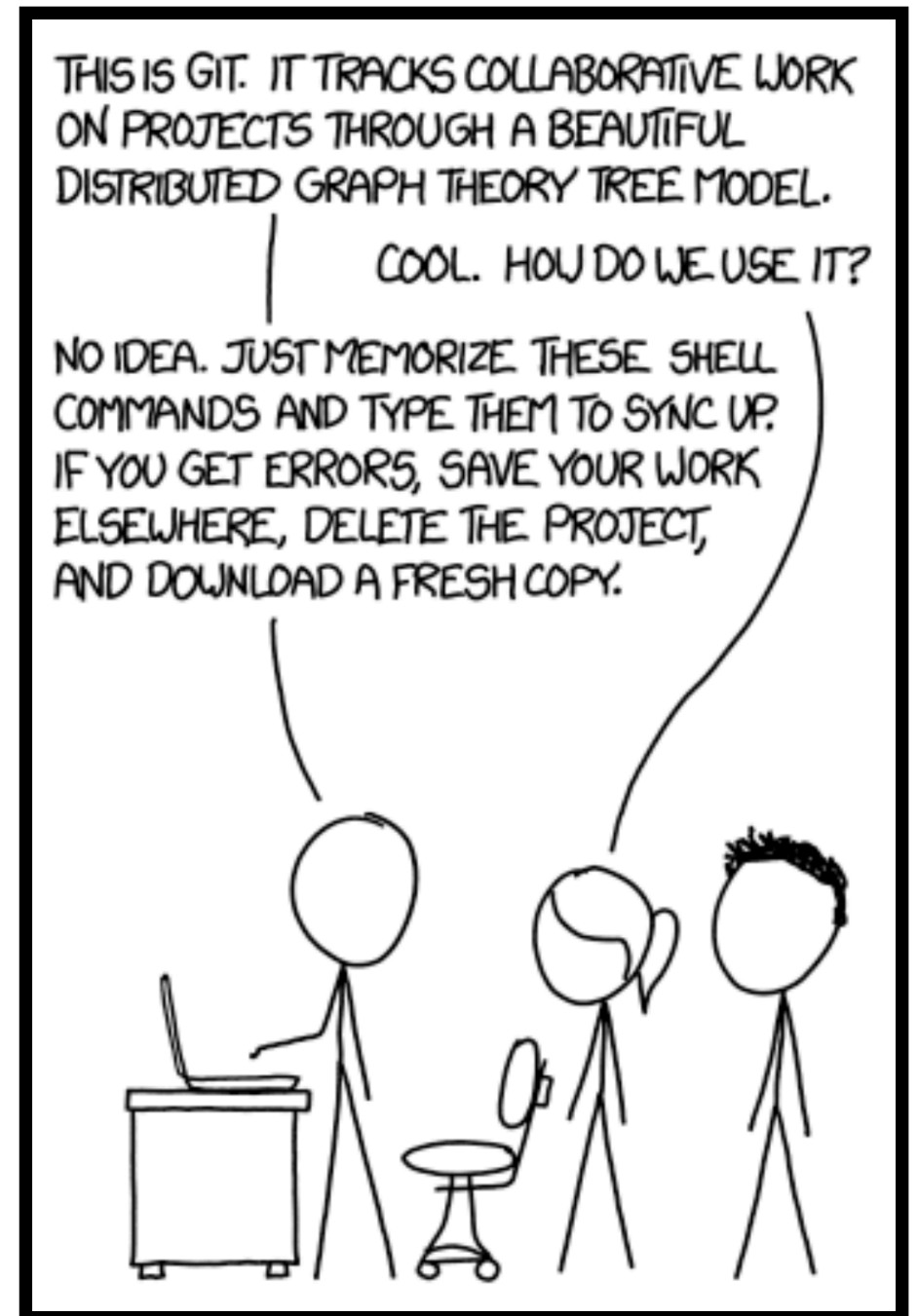
Ognuno risolve qualcosa e rompe qualcos'altro

METODO	VA BENE PER...	...MA
Documento online	tempo reale, salvataggio auto	pessimo per il codice; serve internet
Copiare i file (cp+diff)	qualsiasi file, semplice	disordinato, tutto manuale
File Zip	una versione dell'intera cartella	manuale; difficile estrarre un file
Git	istantanee, recupero, legami, fusioni — in automatico	— (qualche comando da imparare)



Git è un sistema di controllo versione che tiene traccia dei "commit" (istantanee) dei file in un REPOSITORY.

- Git memorizza le vecchie versioni dei file in una cartella nascosta (.git) e le gestisce automaticamente.
- Possiamo dire a Git di tenere traccia di determinati file e di quando effettuare un'istananea.
- Possiamo chiedere a Git di tornare a una vecchia istantanea (anche solo per un singolo file).
- Possiamo chiedere a Git di tenere traccia di chi sta lavorando su cosa, in modo che più persone possano lavorare su cose diverse senza conflitti.



Repository, commit, .git

Repository

la cartella del progetto tracciata da
Git

Commit

un'istantanea salvata dei file in un
dato momento

.git

la cartella nascosta dove Git tiene
tutta la storia

Tu gli dici **quali file** tracciare e **quando** scattare un'istantanea;
lui conserva tutto e, se serve, fonde da solo il lavoro di più persone.

Il flusso base (lineare):

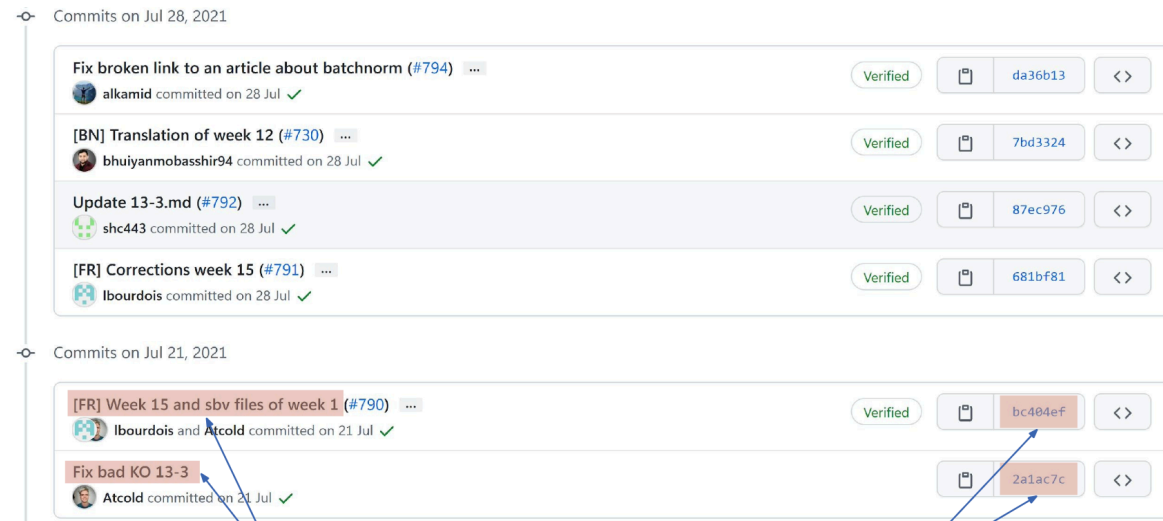
init → add → commit, e si ripete

git init → git add → git commit → modifichi i file ↺

init: attiva Git in una cartella · **add:** "mette in scena" le modifiche da salvare · **commit:** salva un'istantanea della scena

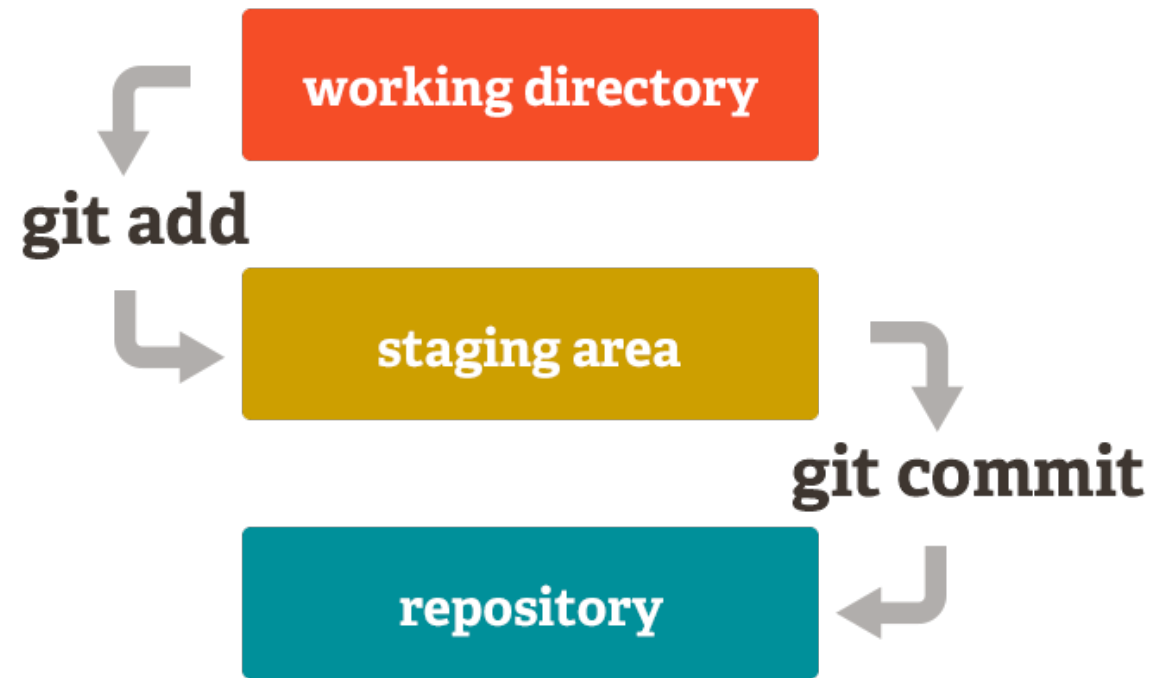
Git si occupa di gestire la storia della repo tramite dei *checkpoint*, che in gergo Git si chiamano **commit**

Ogni commit viene creato dall'utente che sta effettuando modifiche sulla repo



Ogni commit è caratterizzato da un messaggio descrivente le modifiche apportate...

...ed è definito univocamente da un codice numerico esadecimale



Ogni commit è un anello della catena



git add

- Aggiunge file (o modifica file già tracciati) all'area di stage
- Passaggio intermedio prima del commit

1. `git add .` → aggiunge tutti i file modificati e nuovi nella cartella corrente.
2. `git add nomefile` → aggiunge un singolo
3. `git add -A` → aggiunge tutti i file del progetto, inclusi eliminati.

git commit

- Registra in maniera permanente le modifiche
- Crea un “fotogramma” della situazione attuale dei file.
- Ogni commit ha:
 - un hash univoco (es. a3f1c9d...)
 - un **autore**
 - una **data**
 - un messaggio descrittivo

Buone pratiche per il messaggio:

- Deve spiegare **cosa** hai fatto, non come
- Breve ma chiaro (massimo 50–70 caratteri)
- Evitare messaggi generici tipo “update” o “fix”

1. `git commit -m "Messaggio del commit"`

Messaggi di commit

Ogni commit dovrebbe fare **una cosa sola** e lasciare il progetto in uno stato valido.

Messaggi buoni

```
b68f706 Add training trick for handling missing notes
7c0afae Fix audio issue in metronome
fa18a3d Add a metronome for beat tracking
739aa0a Add test for per-millisecond prediction on MIDI files
dc4693a Use last eight piano roll columns to predict next
eb327b4 Fix timing bug in MIDI file parser
b46c6f6 Add MIDI training demo program
a624b32 Switch to cross-entropy loss for MIDI classifier
db3c6f3 Fix bug in MIDI parser
5b0f299 Return effective learning rate from training wrapper
726886d Print more relevant training info from full-piano
↪ predictor
```

Messaggi inutili

```
44e7773 minor
571c20b minor
059cb3f minor
eaa75ae minor
ebbe9db minor
13570e0 minor
3e51470 minor
95a0fad minor
5d2c780 minor
d5caf55 minor
c26b868 minor
080ddf2 minor
f492a3f minor
```

Implementiamo questo workflow in una cartella sul nostro pc:

1. **git init** Crea una nuova repository;
2. **git add [nome_file]** Aggiungi un file al VCS;
3. **git status** Controlla lo stato corrente della repo;
4. **git commit [-m "commit message"]** Fai commit dei file aggiunti (staged);
5. Modifica il file sotto version control (tracked);
6. **git add** Aggiungi le nuove modifiche al VCS;
7. Ripeti!

[**git log** mostra la storia dei commit]

03

I branch: sviluppo in parallelo

Versioni alternative della stessa cartella, per lavorare su più cose insieme.

Branching Workflow

Un branch è una linea di sviluppo indipendente.

Permette ad esempio di lavorare su nuove funzionalità senza toccare subito il codice principale.

Il branch default di una repo si chiama **main**

- parti da un repo "pulito" (niente da committare);
- `git checkout -b feature` crea il branch e ci si sposta;
- lavori come prima: `git add`, `git commit`;
- `git checkout` passa da un branch all'altro.

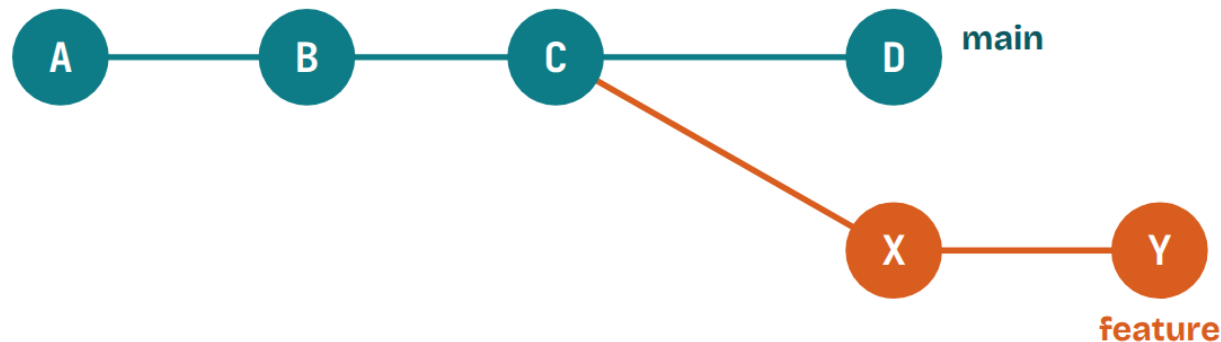


feature parte da un commit di main

Non è una copia dei file

Un branch è un semplice puntatore a un commit, la "punta" di una serie di istantanee.

Per questo crearne uno costa quasi nulla, ed è normale farne di continuo.



- **git branch nome-branch** creazione di un nuovo branch
- **git checkout nome-branch** spostarsi sul branch name_branch
- **git checkout -b nome-branch** creare e spostarsi su un nuovo branch
- **git branch** ottenere una lista di tutti i branch
- **git branch -d nome-branch** rimuovere un branch

04

Combinare i branch

Git merge

Cos'è il merge in Git?

- Il merge è l'operazione che permette di unire i cambiamenti di due branch diversi in uno solo.
- È fondamentale per il lavoro collaborativo e per integrare nuove funzionalità sviluppate su branch paralleli.

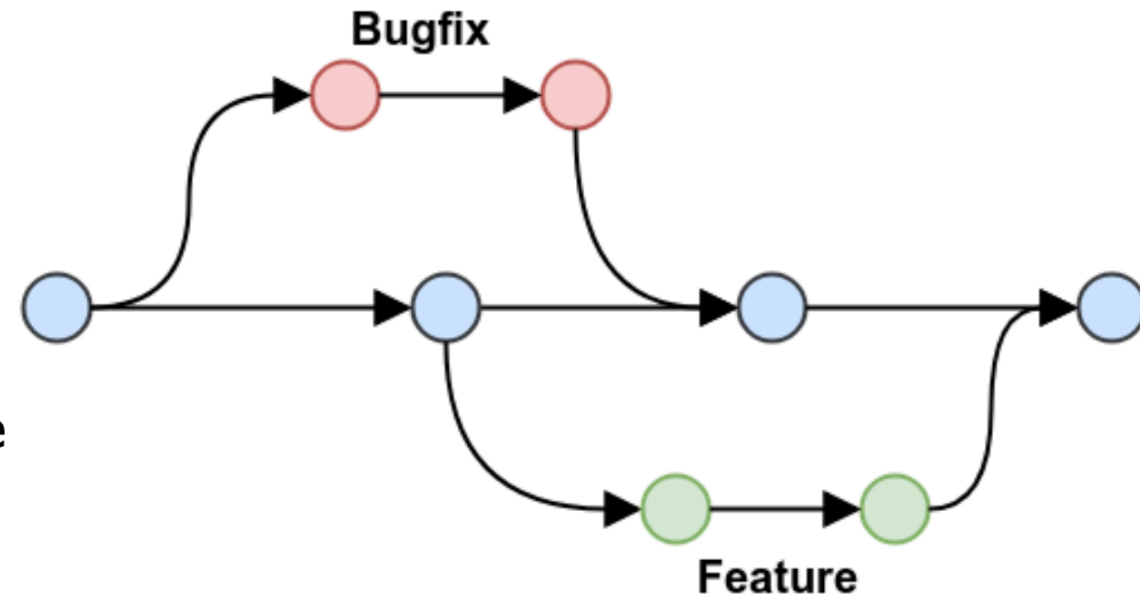
Sintassi di base

```
git merge <branch-da-unire>
```

Esempio

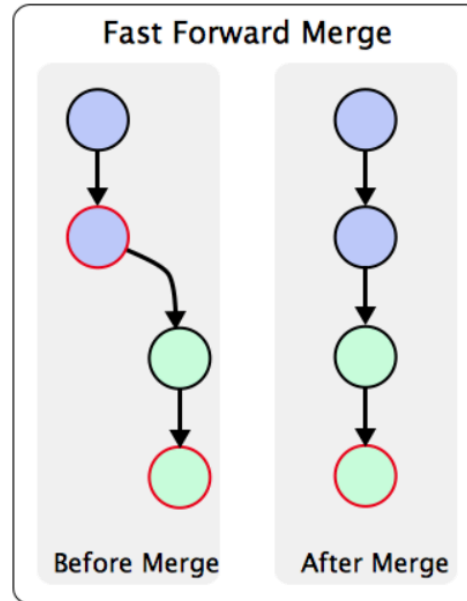
```
git merge feature-branch
```

Unisce il branch feature-branch nel branch attuale



Fast Forward

Il branch è solo più avanti: main lo raggiunge

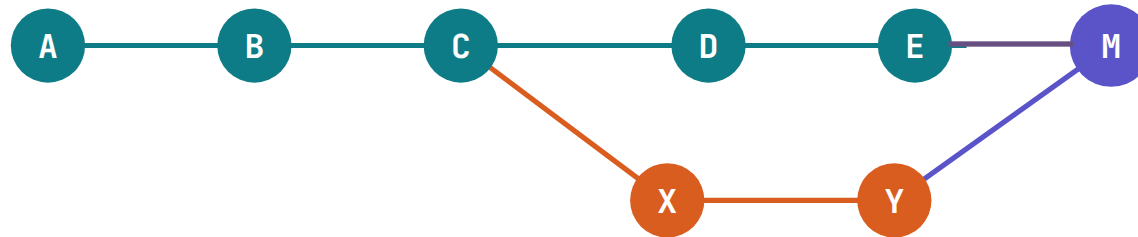


- Avviene quando il branch corrente non ha commit successivi rispetto al branch da unire.
- Il puntatore HEAD viene semplicemente spostato in avanti.

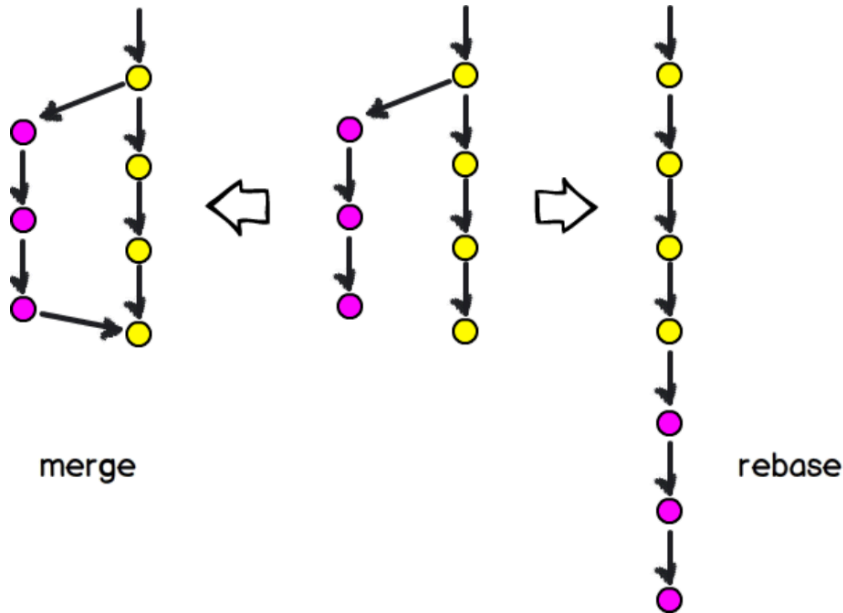
Merge different commit

Entrambi sono andati avanti: serve un commit di fusione

Il merge commit M ha due "genitori" e unisce le due linee.



Rebase



Cosa è il rebase?

Il rebase sposta una serie di commit da un branch sopra un altro. A differenza del merge, non crea un nuovo commit di merge, ma "ricostruisce" la cronologia in modo lineare

Esempio:

Siamo nel branch `feature-branch` e vogliamo riallinearlo con `main`.

```
git checkout feature-branch  
git rebase main
```

Tutti i commit di `feature-branch` verranno applicati sopra `main`, riscrivendo la cronologia.

Rebase vs. Merge

- **Rebase:** Riscrive la cronologia, creando una storia lineare.
- **Merge:** Mantiene la cronologia completa, con commit di merge espliciti.

Quando fare cosa: git branch best practices

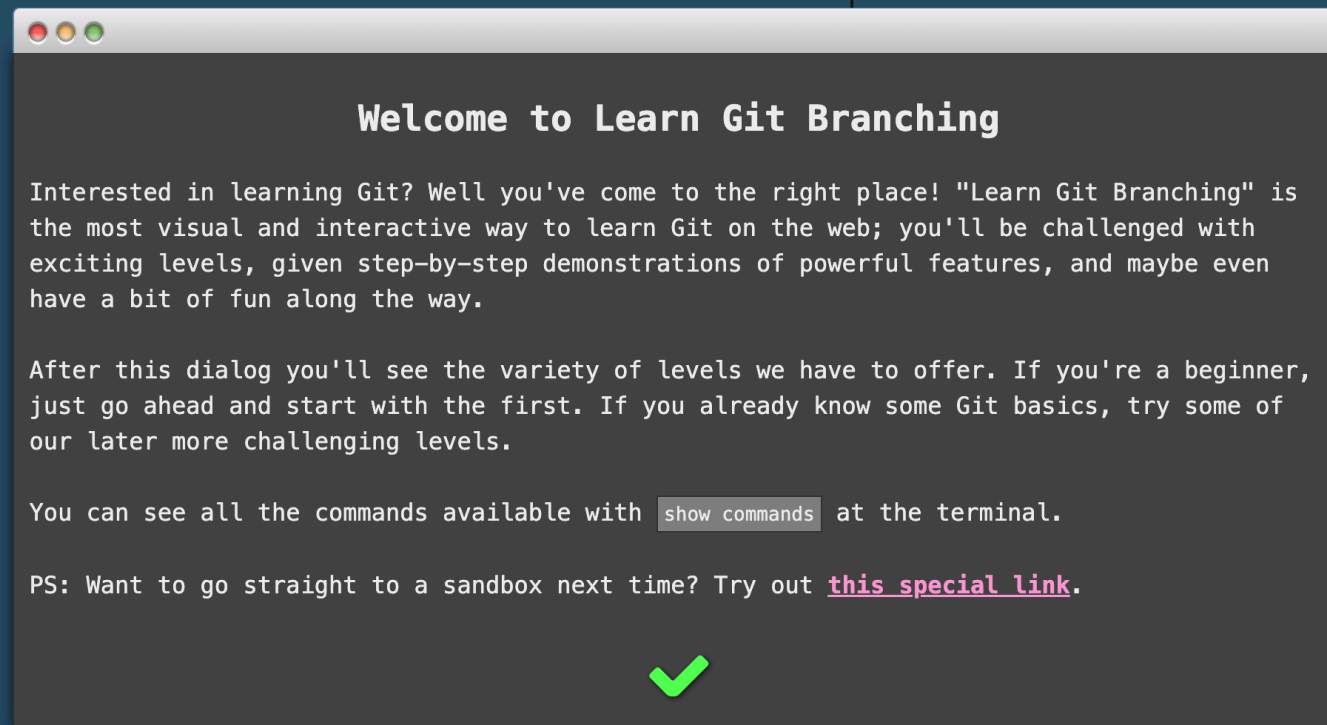
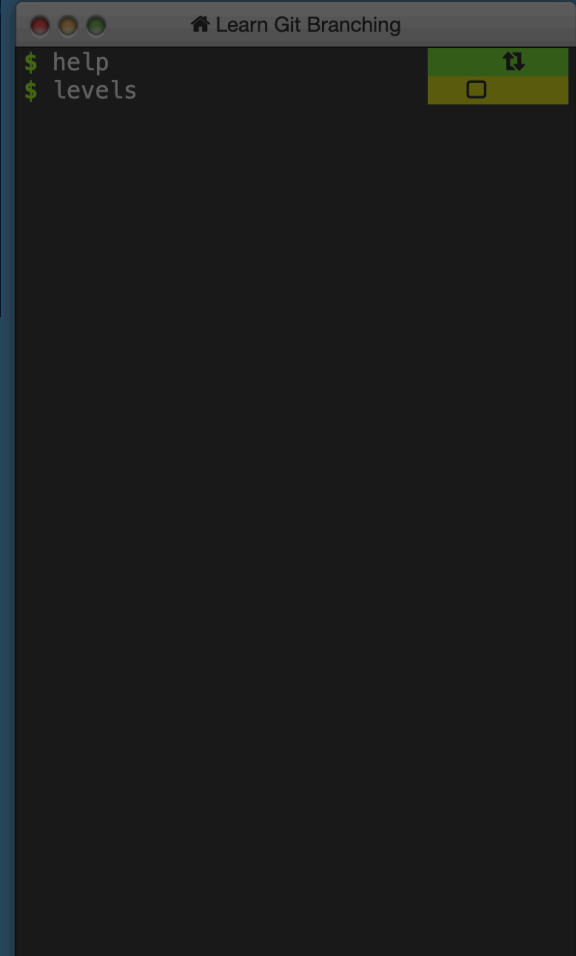
Fast-forward quando è possibile;

Rebase poi fast-forward se sei solo sul branch;

Merge quando i primi due non si possono fare;

Regola d'oro

Non fare **mai** il rebase di un branch che stanno usando anche altre persone: riscrivendo la storia manderesti in confusione il loro lavoro.

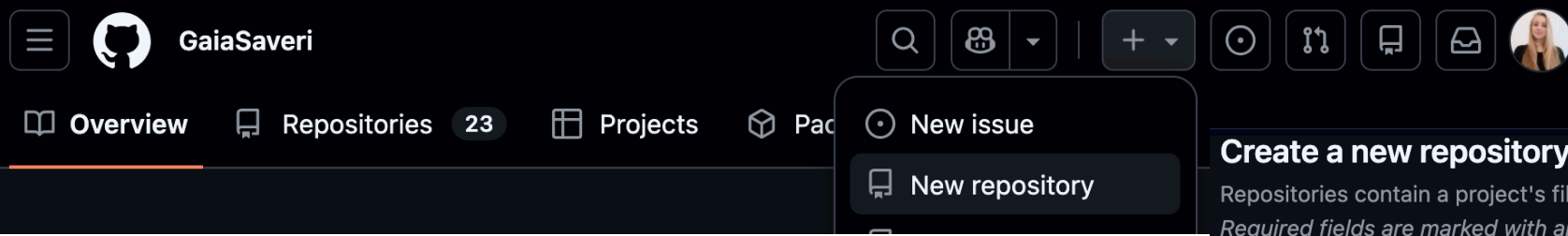


<https://learngitbranching.js.org>

05



Repository remoti



Crea una nuova repo su GitHub

Serve un account GitHub per poter fare queste attività:
crealo se non lo hai già

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#). Required fields are marked with an asterisk (*).

1 General

Owner *



Repository name *

FakeRepo

✓ FakeRepo is available.

Great repository names are short and memorable. How about [congenial-guacamole?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

✓ **Public**

Anyone on the internet can see this repository. You choose who can commit.

Private

You choose who can see and commit to this repository.

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

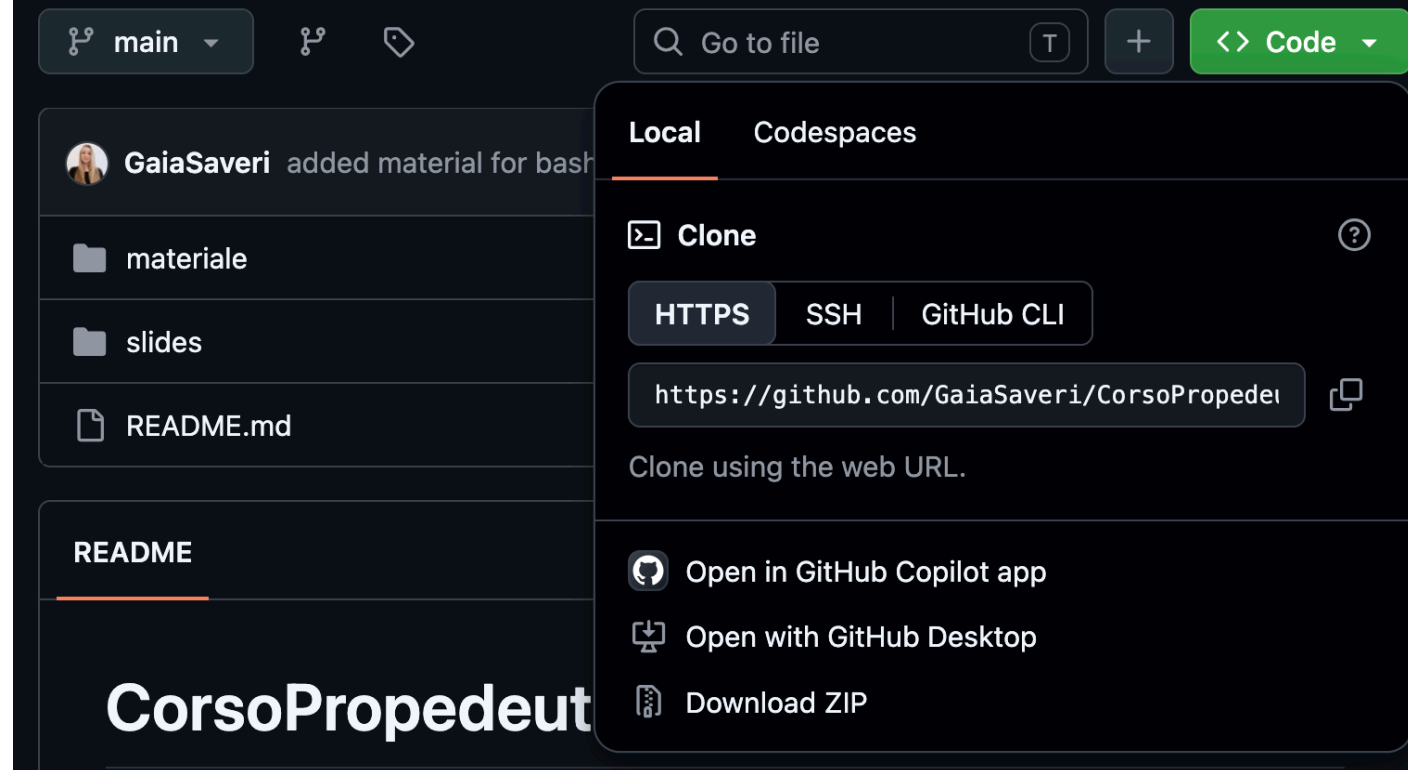
Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Da remoto a locale

Clona, modifica, ricarica



```
# su github.com: crea un repo e copia l'indirizzo HTTPS
$ git clone https://github.com/tuonome/mio-repo.git
$ cd mio-repo
# crea/modifica i file con l'editor, salva, poi:
$ git add .
$ git commit -m "aggiunto README"
$ git push # invia i commit su GitHub
```

git clone

- Crea una **copia completa** di una repository esistente.
- Copia sia i file che tutta la cronologia dei commit.
- Si usa quando vogliamo iniziare a lavorare su un progetto già presente su GitHub (o altrove).

git push

- Invia i commit dal **repository locale** al **repository remoto**.
- Serve per **condividere** il lavoro con altri o salvarlo online.

1. `git push origin main`

`origin` → nome predefinito del repository remoto (GitHub).

`main` → il branch su cui stai lavorando.



git pull

- Aggiorna il repository locale con le modifiche del repository remoto.

1. `git pull origin branch`

Esercizio:

1. Su GitHub crea una nuova repository vuota chiamata: `precorso-git-demo` (niente README).
2. Clonala in locale
3. Crea un `README.md`
4. Fa il commit ("**C1: inizializza repo con README**")
5. Aggiungi un appunto (a tua scelta) su un nuovo file `notes.txt`
6. Fa il commit ("**C2: aggiunta di notes.txt su main**")
7. Crea e passa al branch `feature`
8. Fa il commit ("**F0: creazione del branch feature**")
9. Crea `app.txt` con un testo a scelta
10. Fa il commit ("**F1: aggiunta di app.txt su feature**")
11. Torna su `main`, modifica `README.md`
12. Fa il commit ("**C3: modifica README.md su main**")
13. Esegui il merge del branch `feature` su `main`
14. Stampa l'albero dei commit (`git log --oneline --graph`)
15. Vai su Github e modifica online aggiungendo una riga `README.md`
16. Torna in local e aggiorna

Da locale a remoto

Hai già cartella e file? Collega un repo vuoto

```
$ git init  
$ git add .  
$ git commit -m "primo commit"  
$ git remote add origin https://github.com/tuonome/mio-repo.git  
$ git push -u origin main      # -u collega il branch al remoto
```

Crea repo come prima

Poi il ciclo ricorrente: `git pull` → lavori → `add` → `commit` → `push`.

06

Conflitti, fork e pull request

Cosa succede quando due persone toccano gli stessi file e quale è il modo migliore per collaborare.

Conflitti tra versioni locali della stessa repo

Andrea e Roberta

Roberta va in ferie; Andrea pusha tre commit. Al rientro la copia di Roberta è **indietro**: se prova a push, GitHub lo **rifiuta**. Deve fare prima `git pull`.

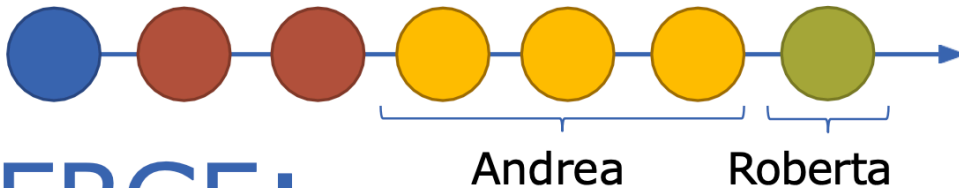
Regola d'oro: fai `git pull` *prima* di lavorare e prima di pushare.

git marca i punti contesi

```
<<<<<< HEAD
la mia versione
=====
la versione dal pull
>>>>>> origin/main
```

Scegli tu cosa tenere, cancella i marcatori, poi `git add` e `git commit`.

Se le modifiche sono compatibili



MERGE:

Git integra i commit di andrea e vi *postpone* il commit di Roberta

Se le modifiche sono incompatibili

Git non sa come integrare il commit di Roberta con quelli di Andrea

Si verifica un cosiddetto **merge conflict**

I merge conflict vanno risolti a mano da Roberta indicando quali parti della versione di Andrea e di quali della versione di Roberta vanno mantenute

Contribuire al progetto di un altro

Fork e Pull Request

Fork

Una copia *personale e remota* del repository di un altro, sul tuo account GitHub. Come un branch, ma in un repository tutto tuo.

Pull Request

Chiedi ai proprietari dell'originale di **integrare** le tue modifiche. È così che si contribuisce all'open-source senza accesso diretto.

Pull Request (PR)

La Pull Request è in generale uno strumento per effettuare un merge di due branch in maniera remota

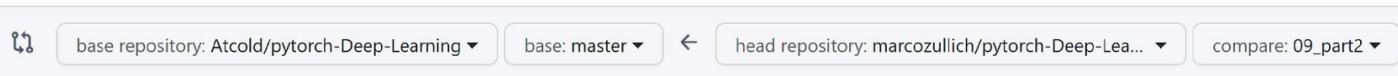


NB: questo significa che le modifiche apportate dal merge devono venir integrate a livello locale con un *pull*

Il branch può provenire sia da un fork della repo che dalla stessa repo



Pull request per merge di due branch (main e ramo) della stessa repo



Pull request per merge di due branch (09_part2 e master) di due repo differenti, una il fork di un'altra